

Devin Fromond

Lab Report

ECE 2031 L04

22 October 2025

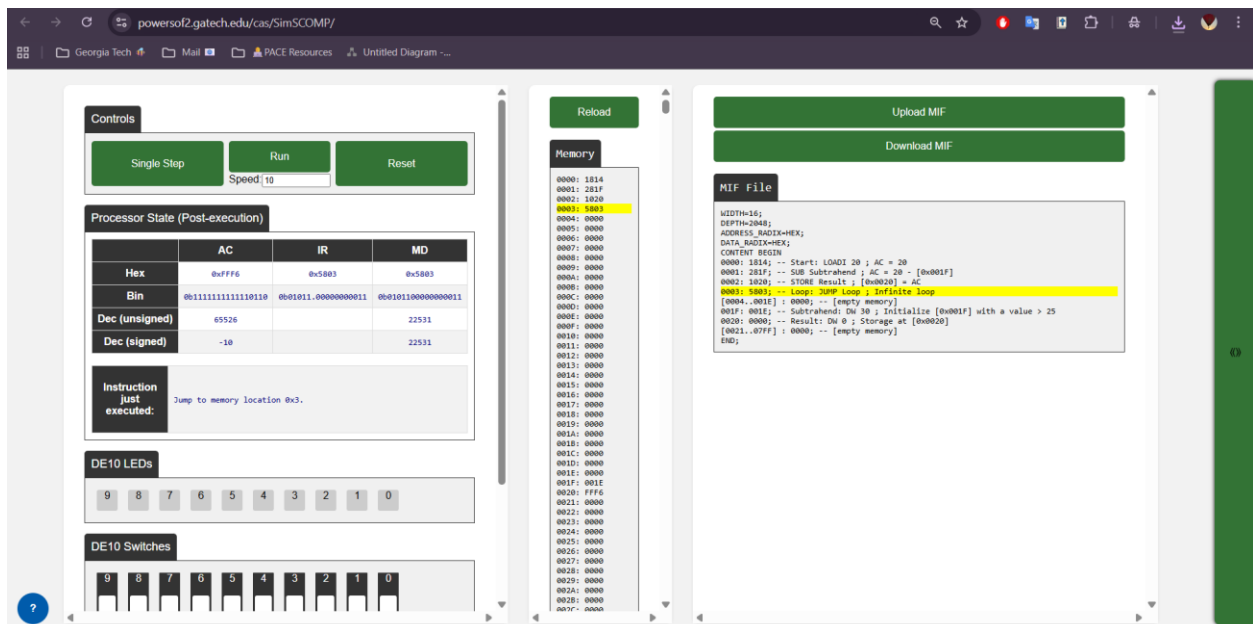


Figure 1. SCOMP processor simulator showing the custom assembly program assembled and executing within the infinite loop, confirming correct operation of the LOADI, SUB, and STORE instructions.

```
; SimpleSubAndStore.asm (ASM)
; Load 20, sub contents of 0x1F, store to 0x20, loop forever
; Devin Fromond
; ECE2031 L4
; 10/18/2025
```

```
ORG      0
Start:    LOADI    20          ; AC = 20
          SUB      Subtrahend ; AC = 20 - [0x001F]
          STORE    Result     ; [0x0020] = AC

Loop:     JUMP     Loop        ; Infinite loop

ORG      &H001F
Subtrahend: DW      30        ; init [0x001F] with a value > 25

          ORG      &H0020
Result:    DW      0          ; Storage at [0x0020]
```

Figure 2. ASM code used to demonstrate functionality of load, subtract, and store operations.

APPENDIX A

VDHL FILE CONTAINING ADDED JPOS/JNZ/SUB OPCODES AND FUNCTIONALITY

```

-- SCOMP.vdh
-- Basic Opcode defs and jpos/jnz/sub functionality
-- Devin Fromond
-- ECE 2031 L04
-- 10/18/2025

-- SCOMP, the Simple Computer.
-- A 16-bit computer designed to be easy to design and modify.
-- Updated 2025

```

```

library altera_mf;
library lpm;
library ieee;

```

```

use altera_mf.altera_mf_components.all;
use lpm.lpm_components.all;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.std_logic_arith.all;

```

```

entity SCOMP is
    port(
        clock      : in      std_logic;
        resetn     : in      std_logic;
        IO_READ    : out     std_logic;
        IO_WRITE   : out     std_logic;
        IO_ADDR    : out     std_logic_vector(10 downto 0);
        IO_DATA    : inout   std_logic_vector(15 downto 0);
        dbg_FETCH  : out     std_logic;
        dbg_AC     : out     std_logic_vector(15 downto 0);
        dbg_PC     : out     std_logic_vector(10 downto 0);
        dbg_NMA    : out     std_logic_vector(10 downto 0);
        dbg_MD     : out     std_logic_vector(15 downto 0);
        dbg_IR     : out     std_logic_vector(15 downto 0)
    );
end SCOMP;

```

```

architecture a of SCOMP is
    type state_type is (
        init, fetch, decode,
        ex_nop, ex_load, ex_store, ex_store2,
        ex_loadi, ex_add, ex_sub, ex_jpos, ex_jnz, ex_addi,
        ex_and, ex_or, ex_xor, ex_shift,
        ex_jump, ex_jneg, ex_jzero,
        ex_return, ex_call,
        ex_in, ex_in2, ex_out, ex_out2
    );

```

```

);

-- custom type for the call stack
constant STACK_SIZE : integer := 10;
type stack_type is array (0 to STACK_SIZE)
of std_logic_vector(10 downto 0);

-- internal signals
signal state : state_type;
signal AC : std_logic_vector(15 downto 0);
signal AC_shifted : std_logic_vector(15 downto 0);
signal PC_stack : stack_type;
signal IR : std_logic_vector(15 downto 0);
signal mem_data : std_logic_vector(15 downto 0);
signal PC : std_logic_vector(10 downto 0);
signal next_mem_addr : std_logic_vector(10 downto 0);
signal operand : std_logic_vector(10 downto 0);
signal MW : std_logic;
signal io_drive_en : std_logic;

begin
-- use altsyncram for unified program and data memory
altsyncram_component : altsyncram
GENERIC MAP (
    numwords_a => 2048,
    widthad_a => 11,
    width_a => 16,
    init_file => "SimpleDemo.mif",
    clock_enable_output_a => "BYPASS",
    lpm_hint => "ENABLE_RUNTIME_MOD=NO",
    intended_device_family => "CYCLONE V",
    clock_enable_input_a => "BYPASS",
    lpm_type => "altsyncram",
    operation_mode => "SINGLE_PORT",
    power_up_uninitialized => "FALSE",
    read_during_write_mode_port_a=>"NEW_DATA_NO_NBE_READ",
    outdata_reg_a => "UNREGISTERED",
    outdata_aclr_a => "NONE",
    width_byteena_a => 1
)
PORT MAP (
    wren_a => MW,
    clock0 => clock,
    address_a => next_mem_addr,
    data_a => AC,
    q_a => mem_data
);

```

```

-- use Intel IP to shift AC for shift instruction
shifter: lpm_clshift
generic map (
    lpm_width      => 16,
    lpm_widthdist  => 4,
    lpm_shiftype   => "arithmetic"
)
port map (
    data      => AC,
    distance  => IR(3 downto 0),
    direction => IR(4),
    result    => AC_shifted
);

-- Memory address comes from PC during fetch
with state select next_mem_addr <=
    PC when fetch,
    operand when others;

-- This makes the operand available immediately after fetch
with state select operand <=
    mem_data(10 downto 0) when decode,
    IR(10 downto 0) when others;

-- use lpm tri-state driver to drive i/o bus
io_bus: lpm_bustri
generic map (
    lpm_width => 16
)
port map (
    data      => AC,
    enabledt  => io_drive_en,
    tridata   => IO_DATA
);

-- IR has the IO address during INs and OUTs
IO_ADDR <= IR(10 downto 0);

process (clock, resetn)
begin
    if (resetn = '0') then                -- Active-low reset
        state <= init;
    elsif (rising_edge(clock)) then
        case state is
            when init =>
                MW      <= '0';    -- clear mem w flag

```

```

PC          <= "000000000000"; -- reset PC
AC          <= x"0000";          -- clear AC
io_drive_en <= '0'; -- don't drive IO
state       <= fetch; -- start cycles

when fetch =>
    io_drive_en <= '0'; -- stop driving IO
    PC          <= PC + 1; -- increment PC
    state       <= decode;

when decode =>
    IR          <= mem_data; -- latch inst
    case mem_data(15 downto 11) is
        when "00000" => -- nop
            state <= ex_nop;
        when "00001" => -- load
            state <= ex_load;
        when "00010" => -- store
            MW      <= '1'; -- mem wr
            state <= ex_store;
        when "00011" => -- loadi
            state <= ex_loadi;
        when "00100" => -- add
            state <= ex_add;
        when "00101" => -- sub
            state <= ex_sub;
        when "00110" => -- addi
            state <= ex_addi;
        when "00111" => -- and
            state <= ex_and;
        when "01000" => -- or
            state <= ex_or;
        when "01001" => -- xor
            state <= ex_xor;
        when "01010" => -- shift
            state <= ex_shift;
        when "01011" => -- jump
            state <= ex_jump;
        when "01100" => -- jneg
            state <= ex_jneg;
        when "01101" => -- jpos
            state <= ex_jpos;
        when "01110" => -- jzero
            state <= ex_jzero;
        when "01111" => -- jnz
            state <= ex_jnz;
        when "10000" => -- call

```

```

        state <= ex_call;
    when "10001" =>          -- return
        state <= ex_return;
    when "10010" =>          -- in
        state <= ex_in;
    when "10011" =>          -- out
        state <= ex_out;
        io_drive_en <= '1';
    when others =>
        state <= ex_nop;
    end case;

when ex_nop =>
    state <= fetch;
when ex_load =>
    AC <= mem_data; -- latch data
    state <= fetch;

when ex_store =>
    MW <= '0'; -- drop MW to end write
    state <= fetch;

when ex_loadi =>          -- sign-extend
    AC <= (IR(10) & IR(10) & IR(10) &
        IR(10) & IR(10) & IR(10 downto 0));
    state <= fetch;

when ex_add =>
    AC <= AC + mem_data; -- addition
    state <= fetch;

when ex_addi =>
    -- sign extension
    AC <= AC + (IR(10) & IR(10) & IR(10) &
        IR(10) & IR(10) & IR(10 downto 0));
    state <= fetch;

when ex_jump =>
    PC <= operand -- overwrite PC
    state <= fetch;

when ex_jneg =>
    if (AC(15) = '1') then
        PC <= operand;
    end if;
    state <= fetch;

```

```

when ex_jzero =>
    if (AC = x"0000") then
        PC    <= operand;
    end if;
    state <= fetch;

when ex_and =>
    AC    <= AC and mem_data; -- AND
    state <= fetch;

when ex_or =>
    AC    <= AC or mem_data;
    state <= fetch;

when ex_xor =>
    AC    <= AC xor mem_data;
    state <= fetch;

when ex_shift =>
    AC    <= AC_shifted;
    state <= fetch;

when ex_call =>
    for i in 0 to STACK_SIZE-1 loop
        PC_stack(i + 1) <= PC_stack(i);
    end loop;
    PC_stack(0) <= PC;
    PC          <= operand;
    state       <= fetch;

when ex_return =>
    for i in 0 to STACK_SIZE-1 loop
        PC_stack(i) <= PC_stack(i + 1);
    end loop;
    PC          <= PC_stack(0);
    state       <= fetch;

when ex_in =>
    IO_READ <= '1'
    state <= ex_in2;

when ex_in2 =>
    IO_READ <= '0';
    AC <= IO_DATA;
    state <= fetch;

when ex_out =>

```

```

        IO_WRITE <= '1';
        state <= ex_out2;

    when ex_out2 =>
        IO_WRITE <= '0';
        state <= fetch;
        -- SUB: AC <= AC - MEM(Address)

    when ex_sub =>
        AC <= AC - mem_data;
        state <= fetch;

    -- JPOS: If AC > 0 , jump.
    when ex_jpos =>
        if (AC(15)='0' and AC/= x"0000") then
            PC <= operand;
        end if;
        state <= fetch;

    -- JNZ: If AC /= 0, jump.
    when ex_jnz =>
        if (AC /= x"0000") then
            PC <= operand;
        end if;
        state <= fetch;

    when others =>
        state <= init;

    end case;
end if;
end process;

-- Additional outputs to aid simulation
dbg_FETCH <= '1' when state = fetch else '0';
dbg_PC <= PC;
dbg_AC <= AC;
dbg_IR <= IR;
dbg_NMA <= next_mem_addr;
dbg_MD <= mem_data;

end a;
```

APPENDIX B

ASM FILE FOR AN SCOMP BYTE-DIFFERENCE PROGRAM

```
; ByteDiff.asm
; Compute |high_byte(Value) - low_byte(Value)| and st in Result
; Devin Fromond
; ECE 2031 L04
; 10/18/2025
```

```
ORG 0
```

```
Start:
```

```
; --- Grab HIGH BYTE into High
LOAD    Value          ; AC = 0xHHLL
AND     HiMask          ; AC = 0xHH00
SHIFT   -8              ; arithmetic >>8 may sign-extend...
AND     LowMask         ; AC = 0x00HH
STORE   High
```

```
; --- Grab LOW BYTE into Low
LOAD    Value          ; AC = 0xHHLL
AND     LowMask         ; AC = 0x00LL
STORE   Low
```

```
; --- Compute (High - Low)
LOAD    High           ; AC = High
SUB     Low             ; AC = High - Low
JNEG    MakePos        ; if negative, negate it
STORE   Result         ; already positive
JUMP    Finish
```

```
MakePos:
```

```
STORE   Diff           ; Diff = (High - Low) < 0
LOADI   0
SUB     Diff            ; AC = -Diff = |High - Low|
STORE   Result
```

```
Finish:
```

```
JUMP    Finish         ; Infinite loop
```

```
Value:      DW    &H89F1      ; Start with 0x89F1 (expected 104)
                                   ; Test Case: HF000 (expected 240)
                                   ; Test Case: HFFFF (expected 0)
                                   ; Test Case: H1248 (expected 54)
```

```
Result:     DW    0
High:       DW    0
Low:        DW    0
Diff:       DW    0
```

```
LowMask:    DW    &H00FF
HiMask:     DW    &HFF0
```